Laboratory Report – Search and A*

Warehouse Robot Navigation, with an LLM as Engineering Assistant

Shrihari Karthik

September 11, 2026

Abstract

This report formulates warehouse robot navigation as a search problem, designs a goal-based search agent, uses a large language model (Claude) to translate that design into Python, and then tests and validates the result. A* with Manhattan distance finds a shortest path of **40** moves on the supplied warehouse while expanding **64** states; breadth-first search finds a path of the same length while expanding the same **64** states. The most instructive finding is negative: on this particular map *every* heuristic tried — Manhattan, zero, Euclidean and $2h$ — expands exactly the same 64 states, because 64 is the total number of free cells and the maze forces the search to reach every one of them. Two supplementary maps were therefore constructed to make the heuristic measurable, and on one of them the inadmissible heuristic $2h$ returns a path of length 16 where the optimum is 12.

Contents

1 Task 0 — Search Problem Formulation **2**
1.1 Specification of $\mathcal{P} = (S, A, T, s_0, G, c)$ . . . 2
1.2 Questions . . . 2

2 Task 1 — Agent Design **3**

3 Task 2 — A* Implementation **4**
3.1 Prompt . . . 4
3.2 Final code . . . 4
3.3 Explanation of the main components . . . 5

4 Task 3 — Testing the Generated Program **5**
4.1 Test 1 — Original warehouse . . . 5
4.2 Test 2 — Trivial case . . . 6
4.3 Test 3 — No solution . . . 6
4.4 Test 4 — Alternative paths . . . 6

5 Task 4 — Inspecting the A* Algorithm **7**
5.1 Questions . . . 7

6 Task 5 — A* Compared with Blind Search **8**
6.1 BFS on the original warehouse . . . 8
6.2 Comparison . . . 9
6.3 Questions . . . 9

7 Task 6 — Investigating the Heuristic **9**
7.1 Why Manhattan distance is appropriate here . . . 9
7.2 Results . . . 10
7.3 Analysis . . . 11

8 Task 7 — Evaluating the LLM-Generated Agent **12**

9 Final Reflection **13**

A Reproducing the Experiments **14**

B Prompts Used with the LLM **15**

C Full Source Code **15**
C.1 astar.py 15
C.2 bfs.py 19

1 Task 0 — Search Problem Formulation

The warehouse is the 17×9 ASCII map supplied in the laboratory sheet. Rows are numbered from the top starting at 0 and columns from the left starting at 0, so `S` $= (1, 1)$ and `G` $= (7, 15)$.

```
##################
#S....#.........#
#.###.#.#######.#
#...#.#.......#.#
###.#.#######.#.#
#...#.........#.#
#.###########.#.#
#.............#G#
##################
```

1.1 Specification of $\mathcal{P} = (S, A, T, s_0, G, c)$

Component	Specification
State S	The set of coordinate pairs (r, c) with $0 \le r < 9$, $0 \le c < 17$ and grid$[r][c] \neq$ `#`. Nothing else is part of the state: the robot carries no orientation, no load and no history. There are 64 such free cells, so $\lvert S \rvert = 64$.
Actions A	$A = \{\text{Up}, \text{Down}, \text{Left}, \text{Right}\}$, corresponding to the displacements $(-1, 0)$, $(+1, 0)$, $(0, -1)$, $(0, +1)$. The action set is the same in every state; validity is handled by the transition function.
Transition T	$T\big((r, c), a\big) = (r + \Delta r_a,\ c + \Delta c_a)$ if that cell lies inside the grid and is not an obstacle, and is undefined otherwise. An action that would leave the grid or enter a `#` cell generates no successor; it is simply not offered.
Initial state s_0	$s_0 = (1, 1)$, the cell marked `S`.
Goal G	$G = \{(7, 15)\}$, the single cell marked `G`. The goal test is the equality $s = (7, 15)$.
Cost c	$c(s, a, s') = 1$ for every applicable action. The cost of a path is therefore the number of moves it contains, and a cheapest path is a shortest path.

1.2 Questions

(a) What information is necessary to specify a state? Only the robot's grid position (r, c). The map itself is static, the robot has no internal state that the actions can change, and the cost of a move does not depend on how the robot arrived. Everything the agent needs in order to decide what to do next, and everything that determines what the future looks like, is contained

in the current cell — this is the Markov property, and it is what makes a bare coordinate pair a sufficient state. Adding the path taken so far would not change the set of reachable futures; it would only multiply the state space and destroy the benefit of the visited set.

(b) What makes an action invalid? Two conditions, both purely geometric. The target cell may lie outside the grid boundary, or it may contain an obstacle `#`. In this map the border is entirely walls, so the boundary condition is never actually the binding one; it is nevertheless checked, because the test maps used later are not all walled in the same way and the agent should not depend on that accident.

(c) Is this a deterministic search problem? Yes. T is a function: a given state and a given valid action always produce exactly one successor, and the outcome is known to the agent before the action is taken. The environment is also fully observable and static. This is what allows offline planning — the whole path can be computed before the robot moves at all. A real warehouse robot with wheel slip, or with other robots moving the obstacles, would break determinism and would require either replanning or a policy rather than a fixed path.

(d) What would constitute a solution? A finite sequence of actions $a_1, \ldots, a_k$ such that applying them in turn from s_0 yields a defined state at every step and the final state is the goal. Equivalently, the corresponding sequence of states $s_0, s_1, \ldots, s_k$ with $s_k \in G$, every s_i free, and every consecutive pair differing by one valid move. An *optimal* solution is one of minimum k, since every action costs 1.

2 Task 1 — Agent Design

This design was written before any prompt was given to the LLM. It is the specification that Prompt 1 (Appendix B) hands over.

1. **State representation.** A Python tuple `(row, col)` of integers. Tuples are hashable, so states can be used directly as dictionary keys and set members, which the visited set and the parent map both require.

2. **Warehouse representation.** A list of strings, one string per row, indexed as `grid[row][col]`. This is the ASCII map essentially unchanged, so the map in the report and the map in the program cannot drift apart. A parsing function scans the grid once to locate `S` and `G` and raises an error if either is missing.

3. **Valid actions.** A fixed list of four (name, displacement) pairs is tried in a fixed order (Up, Down, Left, Right). A successor is generated only if the target cell is in range and is not `#`. Fixing the order is a deliberate choice: it makes runs reproducible, which matters because the number of expanded states is one of the measurements being reported.

4. **Goal detection.** Equality against the goal coordinate. Crucially, the test is applied *when a state is removed from the frontier*, not when it is generated — see Section 5, question (d), for why.

5. **Frontier contents.** A binary min-heap of triples `(f, counter, state)`. `f` is $g(n) + h(n)$ and orders the heap; `counter` is a monotonically increasing insertion number that breaks ties in first-in-first-out order so that equal-f states are expanded in a deterministic sequence; `state` is the coordinate pair. The g values themselves are held in a separate dictionary `g_score` rather than in the heap entry, so that a better g discovered later can be recorded without having to find and rewrite the old heap entry.

6. **Path reconstruction.** A dictionary `came_from` mapping each state to the state it was reached from, plus a parallel dictionary `action_taken` recording the action name. On success, follow the parent pointers back from the goal and reverse the list.

Termination report. The program returns a dictionary with the keys `found`, `path`, `path_length`, `states_expanded` and `actions`. `path_length` counts *actions*, i.e. `len(path)` − 1, and `states_expanded` counts *distinct* states removed from the frontier and expanded, excluding stale duplicate heap entries that are popped and discarded. Both definitions are stated here because they are the measurements the later comparisons depend on, and an ambiguous counter would make those comparisons meaningless.

3 Task 2 — A* Implementation

3.1 Prompt

The initial prompt handed the design above to the LLM verbatim and asked for an A* implementation using the standard library only, together with an explanation of each component. It is reproduced in full as Prompt 1 in Appendix B; the follow-up prompts that produced the final version are Prompts 2–4.

3.2 Final code

The complete program is listed in Appendix C. The core of the search is reproduced here.

```
def astar(text, heuristic=manhattan):
    grid, start, goal = parse_map(text)

    counter = 0
    frontier = [(heuristic(start, goal), counter, start)]

    g_score = {start: 0}
    came_from = {}
    action_taken = {}
    closed = set()          # states already expanded
    states_expanded = 0

    while frontier:
        f, _, current = heapq.heappop(frontier)

        # A state can sit in the heap more than once; skip stale copies.
        if current in closed:
            continue
        closed.add(current)
        states_expanded += 1

        # Goal test on expansion, not on generation: this is what makes the
        # returned path optimal for an admissible heuristic.
        if current == goal:
            path = reconstruct_path(came_from, goal)
            return {
                "found": True,
                "path": path,
                "path_length": len(path) - 1,
                "states_expanded": states_expanded,
                "actions": [action_taken[s] for s in path[1:]],
            }

        for name, nxt in successors(grid, current):
            if nxt in closed:
                continue
            tentative_g = g_score[current] + 1      # every move costs 1
            if tentative_g < g_score.get(nxt, float("inf")):
                g_score[nxt] = tentative_g
                came_from[nxt] = current
                action_taken[nxt] = name
```

```
                counter += 1
                heapq.heappush(
                    frontier, (tentative_g + heuristic(nxt, goal), counter, nxt)
                )

    return {"found": False, "path": None, "path_length": None,
            "states_expanded": states_expanded, "actions": None}
```

3.3 Explanation of the main components

- `parse_map` turns the ASCII text into `(grid, start, goal)`. It is shared by both algorithms, which guarantees that A* and BFS are solving literally the same problem instance.
- `successors` implements T. It yields only valid moves, so the caller never has to think about walls.
- **The frontier** is a `heapq` min-heap. Python's `heapq` compares tuples element by element, so ordering is by f first and by insertion counter second. The counter also guarantees that the third element, the state tuple, is never reached by the comparison — which would otherwise make the expansion order an artefact of coordinate ordering.
- **Lazy deletion.** When a shorter route to an already-queued state is found, the code pushes a new entry rather than decreasing the key of the old one, which `heapq` does not support. The old entry becomes stale; it is recognised on pop because its state is already in `closed`, and is discarded without being counted.
- **The `closed` set** is the visited set. A state enters it at the moment it is expanded. Successors already in `closed` are skipped, which is what stops the search from cycling between two adjacent cells forever.
- **Failure** is reported by falling out of the `while` loop. The loop can only exit that way when the frontier is empty, which means every state reachable from s_0 has been expanded and none was the goal. The state space is finite and no state is expanded twice, so this always happens in finite time — the program cannot loop indefinitely on an unsolvable map.

4 Task 3 — Testing the Generated Program

All four tests are executed by `python3 astar.py`. The outputs below are copied from that run.

4.1 Test 1 — Original warehouse

```
Solution found : yes
Path length    : 40
States expanded: 64
Path:
  (1, 1) -> (1, 2) -> (1, 3) -> (1, 4) -> (1, 5) -> (2, 5) -> (3, 5) -> (4, 5) -> (5, 5) -> (5, 6)
  (5, 7) -> (5, 8) -> (5, 9) -> (5, 10) -> (5, 11) -> (5, 12) -> (5, 13) -> (4, 13) -> (3, 13) -> (3, 12)
  (3, 11) -> (3, 10) -> (3, 9) -> (3, 8) -> (3, 7) -> (2, 7) -> (1, 7) -> (1, 8) -> (1, 9) -> (1, 10)
  (1, 11) -> (1, 12) -> (1, 13) -> (1, 14) -> (1, 15) -> (2, 15) -> (3, 15) -> (4, 15) -> (5, 15) -> (6, 15)
  (7, 15)
Map with path:
##################
#S*****##########
#.#####*#########
#...##*********##
###.#*#########*##
#...#*********##*#
#.###########.#*#
#............#G#
##################
```

Path found	yes
Path length	40 moves (41 cells)
States expanded	64

Two sanity checks on this result. First, Manhattan distance from $(1, 1)$ to $(7, 15)$ is $6+14 = 20$, so the true cost must be at least 20; 40 is consistent, and the factor of two is the cost of the detour the walls impose. Second, the returned path visits 41 of the 64 free cells, every consecutive pair differs by exactly one unit move, every cell on it is free, and the drawn overlay shows a single connected corridor from `S` to `G` — it does not jump walls.

4.2 Test 2 — Trivial case

```
#####
#SG##
#####

Solution found : yes
Path length    : 1
States expanded: 2
Path:
  (1, 1) -> (1, 2)
```

The one-step solution is found. Two states are expanded: the start, and then the goal. This is the minimum possible, and it confirms that the goal test on expansion does not cost an extra layer of search.

4.3 Test 3 — No solution

```
#######
#S....#
###.###
#...#G#
#######

Solution found : NO
Path length    : None
States expanded: 9
```

The goal cell $(3, 5)$ is enclosed by walls on all four sides, so it is not in the connected component of the start. The program terminates and reports failure rather than looping. The 9 expanded states are exactly the free cells reachable from `S` — four in row 1, one in row 2, three in row 3, plus the start itself — which confirms that the search exhausted the reachable component and stopped, rather than failing for some other reason.

4.4 Test 4 — Alternative paths

```
#######
#S....#
#.....#
#....G#
#######

Solution found : yes
Path length    : 6
States expanded: 15
Path:
  (1, 1) -> (2, 1) -> (3, 1) -> (3, 2) -> (3, 3) -> (3, 4) -> (3, 5)
Map with path:
#######
#S....#
#*....#
```

```
#****G#
#######
```

This is an open room, so there are $\binom{6}{2} = 15$ distinct monotone shortest paths from $(1, 1)$ to $(3, 5)$, all of length $2+4 = 6$. The returned path has length 6, so it is one of them — the program returns *a* shortest path, which is all that can be required. (The room also happens to contain 15 free cells, all of which are expanded; the two 15s are a coincidence of this small map, not a relationship.) Which one it returns is decided by the fixed action order and the FIFO tie-break; the important point for validation is the length, and that was independently confirmed against BFS (Section 6).

5 Task 4 — Inspecting the A* Algorithm

Concept	**Where it appears in `astar.py`**
State	The tuple `current`, and the `(r, c)` pairs produced by `parse_map` and `successors`.
Action	The `ACTIONS` list of (name, displacement) pairs; the name is carried through `successors` as `name` and stored in `action_taken`.
Transition	`nxt = (r + dr, c + dc)` inside `successors`, guarded by `is_free(grid, nxt)`. The guard is the partiality of T.
Goal test	`if current == goal:` immediately after the state is popped and added to `closed`.
$g(n)$	The dictionary `g_score`, updated by `tentative_g = g_score[current] + 1`. The `+ 1` is the unit cost function c.
$h(n)$	The `heuristic` parameter, defaulting to `manhattan`. It is called once per generated successor.
$f(n)$	Computed inline as the first element of the pushed tuple: `tentative_g + heuristic(nxt, goal)`.
Frontier	The list `frontier`, maintained as a min-heap by `heapq.heappush` and `heapq.heappop`.
Visited states	The set `closed`, together with the `g_score` dictionary. Both are needed: `closed` records what has been expanded, `g_score` records the best known cost to states that have merely been generated.
Path reconstruction	The `came_from` dictionary and the `reconstruct_path` function, which walks the parent chain backwards and reverses it.

5.1 Questions

(a) What data structure is used for the A* frontier? A binary min-heap (`heapq`) of triples `(f, counter, state)`. Push and pop are both $O(\log n)$. The alternative — scanning a list for the minimum f on every iteration — would be $O(n)$ per expansion and would make the algorithm quadratic in the number of states.

(b) How does the program select the next state to expand? `heapq.heappop` returns the entry with the smallest f; ties go to the entry with the smaller insertion counter. If the popped state is already in `closed` it is a stale duplicate and is discarded without being counted, and the loop pops again. So the state actually expanded is always the unexpanded state with the lowest f value currently known.

(c) Where is the heuristic calculated? In two places, both calls to the `heuristic` parameter: once for the start state when the frontier is initialised, and once for each successor at the moment it is pushed. It is *not* recalculated on pop, and it is not stored — the f value baked into the heap entry is what survives. Making the heuristic a parameter rather than inlining Manhattan distance is what made the Task 6 experiments a matter of passing a different function rather than editing the algorithm.

(d) Does the program explicitly calculate $f(n) = g(n) + h(n)$? Yes, as the expression `tentative_g + heuristic(nxt, goal)` in the `heappush` call. It is worth being explicit about why the goal test is placed on expansion rather than on generation, since the LLM's first version tested on generation. When the goal is first *generated*, the g value attached to it is merely the cost of the first route found to it, and a cheaper route may still be sitting in the frontier. When the goal is *popped*, every remaining frontier entry has $f \geq f(\text{goal}) = g(\text{goal})$ (since $h(\text{goal}) = 0$), and for an admissible h no such entry can lead to a cheaper route to the goal. Only the second placement guarantees optimality. On the warehouse map both placements happen to return length 40, which is exactly why the bug would have survived casual testing.

(e) How does the program prevent unnecessary repeated exploration? Three mechanisms working together. The `closed` set stops a state from being expanded twice. The test `if tentative_g < g_score.get(nxt, inf)` stops a successor from being pushed at all unless the new route to it is strictly better than the best route known, which is what prevents the heap from filling with redundant entries. And the stale-entry check on pop discards superseded heap entries cheaply instead of trying to remove them from the middle of the heap.

6 Task 5 — A* Compared with Blind Search

BFS was implemented in `bfs.py`, reusing `parse_map` and `successors` from `astar.py` so that both algorithms solve an identical problem instance. The frontier is a `collections.deque` used as a FIFO queue and states are marked visited when generated. The warehouse map was not changed.

6.1 BFS on the original warehouse

```
Solution found : yes
Path length    : 40
States expanded: 64
Map with path:
##################
#S****#**********#
#.####*#########*#
#...#*#********#*#
###.#*#########*#*#
#...#**********#*#
#.###########.#*#
#............#G#
##################
```

BFS returns the identical path, not merely a path of the same length.

6.2 Comparison

	Solution found		Path length		States expanded	
Map	BFS	A*	BFS	A*	BFS	A*
Original warehouse	yes	yes	40	40	64	64
Trivial	yes	yes	1	1	2	2
No solution	no	no	–	–	9	9
Alternative paths	yes	yes	6	6	15	15
Supplementary maps, added for Task 6:						
Barrier room	yes	yes	14	14	68	56
Trap room	yes	yes	12	12	39	38

The row required by the laboratory sheet is the first one. The last two rows were added later, once it became clear that the warehouse map does not discriminate between the algorithms at all, and they are discussed in Section 7.

6.3 Questions

(a) Did both algorithms find a solution? Yes, on every solvable map, and both correctly reported failure on the unsolvable one.

(b) Did they find paths of the same length? Yes — 40 on the warehouse, and equal lengths on all six maps. This is the single most useful validation in the whole exercise. With unit costs BFS is guaranteed to return a shortest path, so BFS acts as an independent oracle for A*'s optimality. Any map on which A* returned a longer path than BFS would be direct evidence of a bug (or of an inadmissible heuristic, which is exactly how the $2h$ result in Section 7 was detected). The program runs this cross-check automatically and prints `OK` for all six maps.

(c) Which algorithm expanded fewer states? On the warehouse, *neither* — both expanded exactly 64. On the barrier room A* expanded 56 against BFS's 68, and on the trap room 38 against 39.

(d) Why might A* expand fewer states? BFS orders the frontier by g alone, so it expands states in rings of increasing depth around the start, in every direction equally — it has no information about *where* the goal is. A* orders by $g + h$, so a state that is far from the goal is pushed back behind states of equal depth that are closer to it, and states whose optimistic total cost already exceeds the cost of the solution are never reached at all. The saving is therefore the set of states n with $g(n) + h(n) > C^*$, where C^* is the optimal cost. On the warehouse that set is empty, which is why the two algorithms tie, and that is the subject of the next section.

7 Task 6 — Investigating the Heuristic

7.1 Why Manhattan distance is appropriate here

The robot moves one cell at a time, horizontally or vertically, at cost 1. Any path from $n = (r, c)$ to $G = (r_G, c_G)$ must change the row coordinate by $|r - r_G|$ in total and the column coordinate by $|c - c_G|$ in total, and each single move changes exactly one of the two by exactly one. Therefore every path costs at least $|r - r_G| + |c - c_G|$, which is the Manhattan distance. Formally $h(n) \leq h^*(n)$

for all n: the heuristic is *admissible*. It is also consistent, since a single move changes h by at most 1 and costs exactly 1, so $h(n) \le c(n, n') + h(n')$.

Manhattan distance is the exact cost on an obstacle-free grid, so it is the tightest heuristic obtainable from the relaxed problem "ignore the walls". Euclidean distance is the exact cost of the relaxation "ignore the walls *and* allow diagonal movement"; that relaxation is strictly weaker, so Euclidean distance is still admissible but never larger than Manhattan, and usually smaller. Straight-line distance would be the right choice for a robot that could move in any direction; it is the wrong choice here because it throws away the fact that the robot cannot cut corners.

7.2 Results

Original warehouse.

Heuristic	Solution found	Path length	States expanded
Manhattan	yes	40	64
$h(n) = 0$	yes	40	64
Euclidean	yes	40	64
$2\times$ Manhattan	yes	40	64

All four are identical. My first reaction was that the heuristic parameter was not being used, so I checked it rather than reporting it. The check was to count the free cells in the map: there are exactly **64**. The search therefore expands *every reachable state* no matter which heuristic is supplied, and the counts cannot differ because they are already at their ceiling. The explanation is the structure of the map: it is essentially one long winding corridor with almost no branching, the optimal cost is $C^* = 40$ against a Manhattan estimate of only 20 at the start, and for every free cell n the value $g(n) + h(n)$ stays at or below 40. There is nothing for the heuristic to prune. The result is genuine, and it is arguably the most useful measurement in the laboratory, but it measures the map rather than the heuristic.

Supplementary maps. To obtain a measurement of the heuristic, two further maps were constructed. The first is an open room divided by a central barrier, which gives the search real freedom to wander in the wrong direction. The second contains a direct-looking route that dead-ends, so that a heuristic which over-trusts the straight-line direction is punished.

```
Barrier room
################
#..............#
#.....###.....#
#.....#.#.....#
#..S..#.#..G..#
#.....#.#.....#
#.....###.....#
#..............#
################
```

```
Trap room
############
#S......#.#
#.........#
#.#.......#
#.#.....#.#
#......#.G#
############
```

	Barrier room		Trap room	
Heuristic	Path length	Expanded	Path length	Expanded
$h(n) = 0$	14	68	12	39
Euclidean	14	58	12	38
Manhattan	14	56	12	38
2× Manhattan	14	29	**16**	**19**

7.3 Analysis

$h(n) = 0$**.** A* degenerates to uniform-cost search, which with unit costs is breadth-first search. The path is still optimal — $h = 0$ is trivially admissible — but the search is uninformed and expands the most states of any variant: 68 on the barrier room, against BFS's own 68 on the same map. The two numbers agreeing is itself a consistency check, since the two algorithms should be doing the same thing.

Euclidean. Admissible, optimal, and almost useless here: 58 against Manhattan's 56. Since $\sqrt{\Delta r^2 + \Delta c^2} \leq |\Delta r| + |\Delta c|$, Euclidean distance is a uniformly weaker lower bound, so it dominates less and prunes less. It never returns a worse path, only a slower search.

2× **Manhattan.** This is the informative case. The heuristic is *inadmissible*: $2h$ can exceed h^* whenever a state is closer to the goal than half its true distance, which in these maps is most of them. On the barrier room it happens to get away with it — 29 states expanded against Manhattan's 56, very nearly a halving, for the same length-14 path. On the trap room it does not: it returns a path of length **16** where the optimum is **12**, a 33% worse solution, while expanding only 19 states.

```
Manhattan, length 12
############
#S......#.#
#***.....#
#.########
#.#.....#*#
#......#.G#
############
```

```
2h, length 16
############
#S......#.#
#*.......#
#*#....***#
#*#...**#*#
#******#.G#
############
```

The mechanism is visible in the overlays. Weighting h makes f dominated by the heuristic term, so A* behaves more like greedy best-first search: it dives down the row towards the goal, commits the goal-adjacent states to the closed set with the g values of that first route, and never reopens them when the genuinely cheaper route through the upper corridor is later discovered.

Answering the "Think About It" question. What happens when the heuristic becomes too aggressive is a trade of *optimality* for *speed*, and the trade is silent. Nothing in the output announces that the path is no longer shortest — the program still reports success, still reports a plausible path, and reports a *better*-looking expansion count than the correct version. Without an independent oracle (BFS, here) the degradation is invisible. Formally, with $h \leq w \cdot h^*$ the returned solution is bounded by $w \cdot C^*$; here $w = 2$ and the bound $2 \times 12 = 24$ is comfortably satisfied by the observed 16, but a bound is not a guarantee of optimality. The practical lesson is that for weighted A* the choice of w is an explicit engineering decision about how much solution quality one is willing to sell for search time — not a free improvement.

8 Task 7 — Evaluating the LLM-Generated Agent

1. What parts of the generated code were correct immediately? The overall architecture was right first time, and it was right because the design in Task 1 was handed over as the specification rather than asking for "an A* implementation". The `heapq` frontier, the `g_score` dictionary, the `came_from` parent map, the closed set, the lazy-deletion handling of superseded heap entries, and `reconstruct_path` all arrived in essentially their final form. The Manhattan heuristic was correct. `parse_map` and `successors` were correct, including the bounds check.

2. Did you find any bugs or design problems? Three, in decreasing severity.

1. **Goal test on generation.** The first version returned as soon as the goal appeared as a successor. On unit-cost grids this usually returns the right answer, and it did return 40 on the warehouse, but it is not optimal in general and it is the wrong algorithm. Fixed by testing on pop.

2. **Non-deterministic tie-breaking.** Heap entries were `(f, state)`, so equal-f states were ordered by comparing coordinate tuples. That is not a crash — tuples of integers compare fine — but it makes the expansion order an artefact of coordinate layout rather than of the algorithm, which matters when the expansion count is the quantity being reported. Fixed with the insertion counter.

3. **Ambiguous expansion counter.** The counter was incremented on every pop, including stale duplicates, so "states expanded" was really "heap pops" and would have made the BFS comparison meaningless. Fixed by counting only states that actually enter the closed set.

3. How did you discover those problems? Not by reading the code alone. The tie-breaking and counter issues surfaced when the BFS comparison produced numbers that could not be reconciled with the number of free cells in the map — the counts were larger than the state space, which is impossible for a correct expansion count. The goal-test placement surfaced from asking the LLM to justify its own choice (Prompt 2) rather than asking it whether the code was correct; asked to compare the two placements, it produced the f-ordering argument and the problem with its own version became obvious. The most valuable check of all was the BFS oracle: it is cheap, it is independent, and it is automatic.

4. Did the LLM use terminology or data structures you did not understand? Two things needed looking up. The first was *lazy deletion* — pushing a duplicate entry with a better f instead of decreasing the key of the existing one. This is not how A* is usually written on a whiteboard, and it only makes sense once one knows that `heapq` has no decrease-key operation. The second was the distinction the code draws between `closed` and `g_score`: it was not obvious at first why two structures are needed until I understood that `g_score` also holds entries for states that have been *generated* but not yet expanded.

5. Did you modify the LLM-generated code? Yes. Beyond the three fixes above: the heuristic was lifted out into a parameter with a `HEURISTICS` registry, so Task 6 needed no edits to the algorithm; the action order was fixed explicitly in `ACTIONS` for reproducibility; the return value was made a dictionary with a documented shape so A* and BFS could be compared mechanically; and the `overlay` function was added, which turned out to be the fastest way to see *why* $2h$ was wrong on the trap room.

6. Which tests were most useful? Ranked by what they actually caught: (i) the BFS cross-check, which is the only test that can detect a suboptimal path, and which caught the $2h$ behaviour immediately; (ii) Test 3, the unsolvable map, because termination on failure is the one property that no amount of staring at a successful run will confirm; (iii) counting the free cells, which is not a test of the program at all but of my interpretation of its output, and which converted an apparent bug into the main finding of Task 6. Test 1 on its own — the test that looks most like the task — caught nothing.

7. Could you have trusted the program without testing it? No, and the interesting part is that the untested program would have *looked* fine. It returned length 40 on the warehouse with the goal test in the wrong place; it returned length 40 with an ambiguous expansion counter; and the $2h$ variant returns a perfectly plausible path that is a third too long. Every failure mode encountered in this exercise produced valid-looking output. That is the specific danger: the code is fluent, and fluency is not correctness.

8. What did you understand about A* afterwards that you did not before? That A* is not "BFS plus a distance estimate" but a statement about when it is safe to *commit* to a node. Almost everything that can go wrong — generation-time goal tests, inadmissible heuristics, the closed set without reopening — is the same mistake in different clothing: committing to a state's g value before it is provably minimal. I also understood, from the 64/64 result, that the value of a heuristic is a property of the *map* and not of the heuristic alone. Before running it I would have said Manhattan must beat $h = 0$; on this warehouse it does not, and it cannot.

What was designed, suggested, accepted, changed and tested.

Designed by me	Tasks 0 and 1 in full: the state space, the action set and the partiality of T, the tuple state representation, the choice to keep g out of the heap entry, the parent-pointer reconstruction, and the contents of the termination report.
Suggested by the LLM	The `heapq` frontier with lazy deletion; the `closed`/`g_score` split; `reconstruct_path`; the f-ordering justification for the goal-test placement; the construction principle for the trap map.
Accepted	The data structures and the overall control flow, unchanged.
Changed	Goal test moved to expansion; insertion-counter tie-break added; expansion counter redefined; heuristic made a parameter; action order fixed; return value standardised; overlay printing added.
Tested	Four Task 3 tests; BFS cross-check on all six maps; four heuristics on three maps; the free-cell count used to validate the 64/64 result.

9 Final Reflection

1. Why formulate the search problem before writing the algorithm? Because the formulation is what makes the algorithm's guarantees apply. Deciding that a state is a bare coordinate pair is what licenses the closed set: if the state had included the path taken, two states would never be equal and the visited check would silently do nothing. Deciding that every action costs 1 is what makes "shortest path" and "cheapest path" the same thing, and therefore what makes BFS a valid oracle for A*. Neither decision is visible in the code — they are assumptions the code relies on. In this exercise the formulation also paid off directly as a prompt: handing the LLM a specification produced code that was architecturally right first time, where asking for "A* for a grid" would have produced a plausible program whose assumptions I had not chosen.

2. In what sense is A* "informed"? It is informed in the narrow sense that its frontier ordering uses an estimate of the cost *remaining*, not only the cost already incurred. BFS and uniform-cost search rank a node by $g(n)$, which is entirely backward-looking; A* ranks by $g(n) + h(n)$, where h is knowledge about the goal imported from outside the search — here, the geometry of the grid. The information does not come from the algorithm and it is not learned during the search; it is supplied. That is precisely why the quality of the supply determines the behaviour, and why $h = 0$ collapses A* back into an uninformed algorithm.

3. Why does the choice of heuristic matter? It governs both of A*'s guarantees, and it governs them separately. Admissibility ($h \leq h^*$) controls *correctness*: violate it and the returned path can be too long, as $2h$ demonstrated on the trap room. Tightness — how close h gets to h^* — controls *efficiency*: the states A* can avoid are exactly those with $g(n) + h(n) > C^*$, so a weaker lower bound prunes fewer of them. Euclidean against Manhattan is the clean illustration: both admissible, both optimal, and Euclidean simply slower because it is looser. But the warehouse result is the one worth remembering: when the map offers nothing to prune, an excellent heuristic buys nothing at all. A heuristic is only as valuable as the search space it is allowed to cut away.

4. What did the LLM contribute to the engineering process? Speed on the parts that were already decided, and explanation on the parts that were not. It turned a written design into working code in one step, and it supplied idioms I would have reached for slowly or not at all — lazy deletion in particular. It was most useful when asked to *justify* rather than to produce: the argument that settled the goal-test placement came from a prompt asking it to compare two options and explain which preserves optimality, and the trap map came from asking what property such a map would need rather than asking for a map. It contributed no judgement about whether the results were right. The 64/64 finding, which is the substance of Task 6, came from counting the cells in the map myself, and no amount of prompting would have produced it, because nothing in the code was wrong.

5. What could go wrong if LLM-generated code were accepted untested? The realistic failure is not a crash but a quiet wrong answer. Every defect in this exercise produced output that looked entirely reasonable: the generation-time goal test returned the right length on the test map, the ambiguous counter returned a number, and $2h$ returned a path a third too long while reporting an expansion count that looked like an improvement. A warehouse robot following the $2h$ route would complete every delivery, on a route 33% longer, indefinitely, with nothing in any log to indicate a fault. The harder risk is the one underneath: accepting code means accepting assumptions that were never stated — what counts as a state, when the goal is tested, what the counters mean — and those assumptions are invisible precisely because the code runs. The defence is not to read the generated code more carefully but to hold an independent expectation of what the answer should be, which is what BFS and the free-cell count provided here.

AI Science $\longrightarrow$ AI Engineering

The scientific idea is search. A* is one algorithm implementing it. The LLM is a tool that helps write that algorithm. Responsibility for understanding, testing and validating the result does not transfer with the code.

A Reproducing the Experiments

Python 3, standard library only. No third-party package is required.

```
python3 astar.py    # Task 3 tests, and the Task 6 heuristic tables
python3 bfs.py      # BFS results, Task 5 comparison, optimality cross-check
```

`bfs.py` imports the maps and the shared helpers from `astar.py`, so both files must be in the same directory. All numbers in this report are taken directly from those two runs. The searches are deterministic: the action order is fixed in `ACTIONS` and frontier ties are broken by insertion order, so repeated runs give identical expansion counts.

B Prompts Used with the LLM

The full prompt log is submitted as `prompts.txt`. The nine prompts are, in order: (1) generate A* from the Task 1 design; (2) justify the goal-test placement; (3) make tie-breaking deterministic; (4) define the expansion counter precisely; (5) produce the BFS version; (6) parameterise the heuristic; (7) propose a way to test the surprising 64/64 result; (8) explain the admissibility of Manhattan distance; (9) help construct two discriminating maps.

C Full Source Code

C.1 astar.py

```
"""
A* search for the warehouse robot navigation problem.

Artificial Intelligence -- Laboratory Exercise: Search and A*

Search problem formulation
--------------------------
    S   states        (row, col) integer coordinates of a free cell
    A   actions       Up, Down, Left, Right
    T   transition    (r, c) -> (r + dr, c + dc) if the target cell is
                      inside the grid and is not an obstacle
    s0  initial       the cell marked 'S'
    G   goal          the cell marked 'G'
    c   cost          1 for every action

Running this file executes the full test suite required by Tasks 3 and 6.
"""

import heapq
import math

# ---------------------------------------------------------------------------
# Maps
# ---------------------------------------------------------------------------

WAREHOUSE = """\
##################
#S....#.........#
#.###.#.#######.#
#...#.#.......#.#
###.#.#######.#.#
#...#.........#.#
#.############.#.#
#.............#G#
##################"""

# Test 2: goal immediately adjacent to the start.
TRIVIAL = """\
#####
#SG##
#####"""

# Test 3: the goal is walled off, so no solution exists.
NO_SOLUTION = """\
#######
#S....#
###.###
#...#G#
#######"""
```

```
50
51  # Test 4: an open room, so many distinct shortest paths of length 6 exist.
52  ALTERNATIVE_PATHS = """\
53  #######
54  #S....#
55  #.....#
56  #....G#
57  #######"""
58
59  # Task 6 supplementary map A: an open room split by a central barrier.
60  # Open space lets a heuristic prune, which the warehouse maze does not.
61  BARRIER_ROOM = """\
62  ###############
63  #.............#
64  #.....###.....#
65  #.....#.#.....#
66  #..S..#.#..G..#
67  #.....#.#.....#
68  #.....###.....#
69  #.............#
70  ###############"""
71
72  # Task 6 supplementary map B: the direct-looking route is a trap.
73  # An inadmissible heuristic commits to it and returns a sub-optimal path.
74  TRAP_ROOM = """\
75  ############
76  #S......#.#
77  #.........#
78  #.#.......#
79  #.#.....#.#
80  #......#.G#
81  ############"""
82
83  # ---------------------------------------------------------------------------
84  # Environment representation
85  # ---------------------------------------------------------------------------
86
87  # Fixed action order, so runs are reproducible.
88  ACTIONS = [
89      ("Up", (-1, 0)),
90      ("Down", (1, 0)),
91      ("Left", (0, -1)),
92      ("Right", (0, 1)),
93  ]
94
95
96  def parse_map(text):
97      """Return (grid, start, goal) for an ASCII warehouse map.
98
99      grid  -- list of strings, grid[row][col] is one cell
100     start -- (row, col) of 'S'
101     goal  -- (row, col) of 'G'
102     """
103     grid = [line for line in text.splitlines() if line]
104     start = goal = None
105     for r, row in enumerate(grid):
106         for c, cell in enumerate(row):
107             if cell == "S":
108                 start = (r, c)
109             elif cell == "G":
110                 goal = (r, c)
111     if start is None:
112         raise ValueError("map has no start cell 'S'")
113     if goal is None:
114         raise ValueError("map has no goal cell 'G'")
115     return grid, start, goal
116
117
118 def is_free(grid, state):
119     """True if `state` is inside the grid and is not an obstacle."""
120     r, c = state
121     if r < 0 or r >= len(grid):
122         return False
123     if c < 0 or c >= len(grid[r]):
```

```
        return False
    return grid[r][c] != "#"


def successors(grid, state):
    """Yield (action_name, next_state) for every *valid* action in `state`."""
    r, c = state
    for name, (dr, dc) in ACTIONS:
        nxt = (r + dr, c + dc)
        if is_free(grid, nxt):
            yield name, nxt


# ---------------------------------------------------------------------
# Heuristics
# ---------------------------------------------------------------------


def manhattan(state, goal):
    """h(n) = |x - x_G| + |y - y_G|.  Admissible for 4-way unit-cost movement."""
    return abs(state[0] - goal[0]) + abs(state[1] - goal[1])


def zero(state, goal):
    """h(n) = 0.  A* degenerates to uniform-cost search."""
    return 0


def euclidean(state, goal):
    """Straight-line distance.  Admissible but weaker than Manhattan here."""
    return math.hypot(state[0] - goal[0], state[1] - goal[1])


def double_manhattan(state, goal):
    """h(n) = 2 x Manhattan.  Inadmissible: it can overestimate."""
    return 2 * manhattan(state, goal)


HEURISTICS = {
    "manhattan": manhattan,
    "zero": zero,
    "euclidean": euclidean,
    "double_manhattan": double_manhattan,
}


# ---------------------------------------------------------------------
# A* search
# ---------------------------------------------------------------------


def reconstruct_path(came_from, goal):
    """Walk the parent pointers back from the goal and reverse the result."""
    path = [goal]
    while path[-1] in came_from:
        path.append(came_from[path[-1]])
    path.reverse()
    return path


def astar(text, heuristic=manhattan):
    """Run A* on an ASCII map.

    Returns a result dictionary with the keys required by Task 1:
        found, path, path_length, states_expanded, actions
    `path_length` counts *actions*, i.e. len(path) - 1.
    """
    grid, start, goal = parse_map(text)

    # The frontier is a min-heap ordered by f(n) = g(n) + h(n).
    # `counter` breaks ties in FIFO order so that runs are deterministic.
    counter = 0
    frontier = [(heuristic(start, goal), counter, start)]

```

```
    g_score = {start: 0}
    came_from = {}
    action_taken = {}
    closed = set()          # states already expanded
    states_expanded = 0

    while frontier:
        f, _, current = heapq.heappop(frontier)

        # A state can sit in the heap more than once; skip stale copies.
        if current in closed:
            continue
        closed.add(current)
        states_expanded += 1

        # Goal test on expansion, not on generation: this is what makes the
        # returned path optimal for an admissible heuristic.
        if current == goal:
            path = reconstruct_path(came_from, goal)
            return {
                "found": True,
                "path": path,
                "path_length": len(path) - 1,
                "states_expanded": states_expanded,
                "actions": [action_taken[s] for s in path[1:]],
            }

        for name, nxt in successors(grid, current):
            if nxt in closed:
                continue
            tentative_g = g_score[current] + 1      # every move costs 1
            if tentative_g < g_score.get(nxt, float("inf")):
                g_score[nxt] = tentative_g
                came_from[nxt] = current
                action_taken[nxt] = name
                counter += 1
                heapq.heappush(
                    frontier, (tentative_g + heuristic(nxt, goal), counter, nxt)
                )

    # Frontier exhausted: every reachable state was expanded and none was the
    # goal, so the problem has no solution.
    return {
        "found": False,
        "path": None,
        "path_length": None,
        "states_expanded": states_expanded,
        "actions": None,
    }


# ---------------------------------------------------------------------------
# Reporting helpers
# ---------------------------------------------------------------------------


def format_path(path, per_line=10):
    """Render a coordinate path over several lines so it stays readable."""
    if path is None:
        return "  (none)"
    chunks = []
    for i in range(0, len(path), per_line):
        chunks.append("  " + " -> ".join(str(p) for p in path[i:i + per_line]))
    return "\n".join(chunks)


def overlay(text, path):
    """Return the map with the solution path drawn using '*'."""
    grid = [list(line) for line in text.splitlines() if line]
    if path:
        for r, c in path[1:-1]:
            grid[r][c] = "*"
    return "\n".join("".join(row) for row in grid)

```

```

def report(name, text, result, show_map=True, show_path=True):
    print("=" * 62)
    print(name)
    print("=" * 62)
    print("Solution found : %s" % ("yes" if result["found"] else "NO"))
    print("Path length    : %s" % result["path_length"])
    print("States expanded: %d" % result["states_expanded"])
    if result["found"] and show_path:
        print("Path:")
        print(format_path(result["path"]))
    if result["found"] and show_map:
        print("Map with path:")
        print(overlay(text, result["path"]))
    print()


# ---------------------------------------------------------------------------
# Task 3 and Task 6 experiments
# ---------------------------------------------------------------------------


def run_tests():
    print("### TASK 3: TESTING THE GENERATED PROGRAM ###\n")
    report("Test 1 -- Original warehouse", WAREHOUSE, astar(WAREHOUSE))
    report("Test 2 -- Trivial case (goal adjacent to start)",
           TRIVIAL, astar(TRIVIAL))
    report("Test 3 -- No solution (goal walled off)",
           NO_SOLUTION, astar(NO_SOLUTION))
    report("Test 4 -- Alternative paths (open room)",
           ALTERNATIVE_PATHS, astar(ALTERNATIVE_PATHS))

    print("### TASK 6: HEURISTIC INVESTIGATION ###\n")
    maps = [
        ("Original warehouse", WAREHOUSE),
        ("Barrier room (supplementary)", BARRIER_ROOM),
        ("Trap room (supplementary)", TRAP_ROOM),
    ]
    for map_name, text in maps:
        print(map_name)
        print("  %-18s %-8s %-13s %s" % ("heuristic", "found", "path length",
                                         "states expanded"))
        print("  " + "-" * 56)
        for label in ("manhattan", "zero", "euclidean", "double_manhattan"):
            res = astar(text, HEURISTICS[label])
            print("  %-18s %-8s %-13s %s" % (
                label,
                "yes" if res["found"] else "NO",
                res["path_length"],
                res["states_expanded"],
            ))
        print()

    # The inadmissible heuristic is the interesting case: show the two paths.
    print("Trap room -- path returned by admissible Manhattan (length %d):"
          % astar(TRAP_ROOM, manhattan)["path_length"])
    print(overlay(TRAP_ROOM, astar(TRAP_ROOM, manhattan)["path"]))
    print("Trap room -- path returned by inadmissible 2h (length %d):"
          % astar(TRAP_ROOM, double_manhattan)["path_length"])
    print(overlay(TRAP_ROOM, astar(TRAP_ROOM, double_manhattan)["path"]))
    print()


if __name__ == "__main__":
    run_tests()
```

C.2 bfs.py

```
"""
Breadth-first search for the warehouse robot navigation problem.

Artificial Intelligence -- Laboratory Exercise: Search and A*
```

```
5
6   This is the blind-search counterpart to astar.py, used for the Task 5
7   comparison.  The search problem is identical -- same map, same states,
8   same actions, same unit costs.  Only the frontier discipline changes:
9
10      A*   min-heap ordered by f(n) = g(n) + h(n)
11      BFS  FIFO queue, no cost or heuristic information at all
12
13  Because every action costs 1, BFS is also guaranteed to return a
14  shortest path, which makes it a useful oracle for checking A*.
15
16  Running this file prints the BFS results and the Task 5 comparison table.
17  """
18
19  from collections import deque
20
21  from astar import (
22      WAREHOUSE,
23      TRIVIAL,
24      NO_SOLUTION,
25      ALTERNATIVE_PATHS,
26      BARRIER_ROOM,
27      TRAP_ROOM,
28      astar,
29      parse_map,
30      successors,
31      reconstruct_path,
32      report,
33  )
34
35
36  def bfs(text):
37      """Run breadth-first graph search on an ASCII map.
38
39      Returns the same result dictionary shape as astar(), so the two can be
40      compared directly.
41      """
42      grid, start, goal = parse_map(text)
43
44      frontier = deque([start])
45      visited = {start}            # states generated (BFS never needs to revisit)
46      came_from = {}
47      action_taken = {}
48      states_expanded = 0
49
50      while frontier:
51          current = frontier.popleft()
52          states_expanded += 1
53
54          # BFS may safely test for the goal on expansion; with unit costs the
55          # first time the goal is dequeued it is already at minimum depth.
56          if current == goal:
57              path = reconstruct_path(came_from, goal)
58              return {
59                  "found": True,
60                  "path": path,
61                  "path_length": len(path) - 1,
62                  "states_expanded": states_expanded,
63                  "actions": [action_taken[s] for s in path[1:]],
64              }
65
66          for name, nxt in successors(grid, current):
67              if nxt not in visited:
68                  visited.add(nxt)
69                  came_from[nxt] = current
70                  action_taken[nxt] = name
71                  frontier.append(nxt)
72
73      return {
74          "found": False,
75          "path": None,
76          "path_length": None,
77          "states_expanded": states_expanded,
78          "actions": None,
```

```
    }


def compare(name, text):
    """Print one row of the Task 5 comparison table."""
    b = bfs(text)
    a = astar(text)
    print("%-26s | %-9s %-9s | %-9s %-9s | %-9s %-9s" % (
        name,
        "yes" if b["found"] else "NO",
        "yes" if a["found"] else "NO",
        b["path_length"], a["path_length"],
        b["states_expanded"], a["states_expanded"],
    ))
    return b, a


def run_comparison():
    print("### BFS ON THE ORIGINAL WAREHOUSE ###\n")
    report("BFS -- Original warehouse", WAREHOUSE, bfs(WAREHOUSE))

    print("### TASK 5: BFS vs A* ###\n")
    header = "%-26s | %-19s | %-19s | %-19s" % (
        "map", "solution found", "path length", "states expanded")
    print(header)
    print("%-26s | %-9s %-9s | %-9s %-9s | %-9s %-9s" % (
        "", "BFS", "A*", "BFS", "A*", "BFS", "A*"))
    print("-" * len(header))
    results = {
        "Original warehouse": compare("Original warehouse", WAREHOUSE),
        "Trivial": compare("Trivial", TRIVIAL),
        "No solution": compare("No solution", NO_SOLUTION),
        "Alternative paths": compare("Alternative paths", ALTERNATIVE_PATHS),
        "Barrier room": compare("Barrier room", BARRIER_ROOM),
        "Trap room": compare("Trap room", TRAP_ROOM),
    }
    print()

    # Cross-check: on unit-cost maps BFS is optimal, so A* must match it.
    print("### OPTIMALITY CROSS-CHECK (A* length == BFS length?) ###\n")
    for name, (b, a) in results.items():
        ok = (b["found"] == a["found"]) and (b["path_length"] == a["path_length"])
        print("%-26s %s" % (name, "OK" if ok else "MISMATCH"))
    print()


if __name__ == "__main__":
    run_comparison()
```